DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 2 – Project Report

ENERGY-EFFICIENT IoT SENSOR NETWORK ROUTING VISUALIZER

GUI-Based Implementation and Analysis of Prim's Algorithm ($O(N^2)$) for Minimum Power IoT Topologies

Submitted by

JYOTHIS PRAMOD

Register No: AJC24CC019

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS)

Kanjirappally – 686518, Kerala

APJ Abdul Kalam Technological University

November 2026

ABSTRACT

Battery-powered sensor nodes in an Internet of Things (IoT) deployment spend most of their energy on radio transmission. When every node communicates directly with a central gateway, distant nodes burn power rapidly and the network's lifetime collapses. This project models an IoT sensor field as a complete weighted graph in which every vertex is a sensor and every edge weight is the transmission power (in milliwatts) needed to link two sensors. The problem of connecting all sensors with the least total transmission power is therefore the Minimum Spanning Tree (MST) problem.

The application, IoT Energy Router, is a desktop GUI that implements Prim's algorithm using an adjacency matrix and a linear key array, giving $O(N^2)$ time — the asymptotically best practical choice for dense, fully connected graphs such as wireless sensor fields. Users can build a topology, choose a gateway (start node), and run the algorithm either step by step or in one pass. At each step the tool highlights the chosen minimum-energy link, shows the two next-best alternate links with their power penalty, and reports live energy-savings metrics.

On the six-node preset network, the MST connects all sensors at 117.60 mW, compared with 350.80 mW for direct gateway links — a saving of 233.20 mW (66.5%). The report presents the problem formulation, algorithm design, correctness argument, complexity analysis, implementation, step-wise results and testing of the system.

Keywords: Prim's Algorithm, Minimum Spanning Tree, IoT, Wireless Sensor Networks, Energy-Efficient Routing, Greedy Algorithms, Algorithm Visualization

TABLE OF CONTENTS

ABSTRACT 1
1. INTRODUCTION 4
1.1 Background 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope of the Project 4
1.5 Organisation of the Report 5
2. LITERATURE REVIEW AND THEORETICAL BACKGROUND 6
2.1 Energy Consumption in Wireless Sensor Networks 6
2.2 Graph Representation of Sensor Networks 6
2.3 Minimum Spanning Trees 6
2.4 Prim's Algorithm 6
2.5 Kruskal's Algorithm 6
2.6 Why Prim's Algorithm for IoT Topologies 7
3. PROBLEM FORMULATION 8
3.1 Graph Model 8
3.2 Objective Function 8
3.3 Baseline for Comparison 8
3.4 Input and Output 8
4. ALGORITHM DESIGN 9
4.1 Design Strategy: Greedy Method 9
4.2 Data Structures 9
4.3 Algorithm Steps 9
4.4 Pseudocode 9
4.5 Proof of Correctness (Cut Property) 10
4.6 Alternate Path Penalty 10
5. SYSTEM DESIGN AND IMPLEMENTATION 11
5.1 System Architecture 11
5.2 Development Environment 11
5.3 GUI Modules 11
5.4 Visual Encoding 12
5.5 Core Implementation 12
6. COMPLEXITY ANALYSIS 14
6.1 Time Complexity 14
6.2 Space Complexity 14
6.3 Comparison with Alternative Approaches 14
6.4 Growth of Operations 15
7. RESULTS AND DISCUSSION 16
7.1 Initial Topology 16
7.2 Step-wise Construction of the MST 17
7.3 Execution Trace 19
7.4 Energy Savings 20
7.5 Discussion 20
8. TESTING 21
8.1 Testing Approach 21
8.2 Test Cases 21
8.3 Correctness Verification 21
9. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE 22

9.1 Applications 22
9.2 Limitations 22
9.3 Future Enhancements 22
10. CONCLUSION 23
REFERENCES 24

1. INTRODUCTION

1.1 Background

The Internet of Things connects large numbers of small, low-cost devices — temperature probes, soil-moisture sensors, structural-health monitors, smart meters — that sense their environment and forward data to a gateway. These devices usually run on batteries or energy harvesting and are often deployed in places where replacing a battery is expensive or impossible. In such Wireless Sensor Networks (WSNs), the radio is the dominant consumer of energy, and the power needed to transmit grows quickly with distance.

A naïve design in which every sensor talks directly to the gateway (a star topology) forces far-away nodes to transmit at high power. A better design lets nodes relay data through their neighbours, so that every hop is short. Choosing which links to keep so that the whole network stays connected while total transmission power is minimised is a classic graph-optimisation problem: finding a Minimum Spanning Tree.

1.2 Problem Statement

Given N IoT sensor nodes at known positions and a transmission-power cost for every pair of nodes, find a set of $N - 1$ links that connects every sensor to the gateway (directly or through relays) such that the total transmission power of the network is minimum. In addition, build an interactive graphical tool that visualises how Prim's algorithm constructs this topology step by step and quantifies the energy saved compared with direct links.

1.3 Objectives

- Model an IoT sensor field as a complete, undirected, weighted graph with edge weights in milliwatts.
- Implement Prim's algorithm in its $O(N^2)$ array-based form, suited to dense graphs.
- Develop a GUI that lets users build, edit and load sensor topologies and select the gateway node.
- Visualise each greedy decision, including the chosen link and the next-best alternatives with their power penalty.
- Compute and display energy-savings metrics (MST cost vs. direct-link cost).
- Analyse the time and space complexity and compare it with alternative MST approaches.

1.4 Scope of the Project

The project focuses on the topology-construction step of energy-aware routing: deciding which sensor-to-sensor links form the backbone. It treats edge cost as static transmission power derived from node positions. Real-world factors such as packet collisions, interference,

duty cycling and residual battery levels are outside the scope, though they are discussed under future enhancements.

1.5 Organisation of the Report

Chapter 2 reviews the theory behind MSTs and energy in sensor networks. Chapter 3 formulates the problem as a graph. Chapter 4 explains the algorithm design and its correctness. Chapter 5 describes the system design and implementation. Chapter 6 presents complexity analysis. Chapter 7 discusses results with screenshots, Chapter 8 covers testing, Chapter 9 lists applications, limitations and future scope, and Chapter 10 concludes the report.

2. LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 Energy Consumption in Wireless Sensor Networks

In a sensor node, energy is consumed by sensing, processing and communication, with communication being by far the largest component. According to the widely used first-order radio model, the energy to transmit a packet grows with the square of the distance for short ranges (free-space model) and with the fourth power for longer ranges (multipath model). Consequently, replacing one long link with several short hops can reduce total energy use significantly. This principle underlies most energy-efficient WSN routing schemes.

2.2 Graph Representation of Sensor Networks

A sensor network is naturally represented as a graph G = (V, E), where V is the set of sensor nodes and E is the set of feasible communication links. Each edge (u, v) carries a weight w(u, v) representing the cost of communication — here, transmission power. Because every sensor can, in principle, reach every other sensor by raising its transmit power, the graph is complete: $|E| = N(N - 1)/2$.

2.3 Minimum Spanning Trees

A spanning tree of a connected graph is a subset of edges that connects all vertices without forming any cycle; it always has exactly $N - 1$ edges. A Minimum Spanning Tree is a spanning tree whose total edge weight is the smallest possible. MSTs are used in network design, clustering, circuit layout and approximation algorithms. The two classic greedy algorithms for MSTs are Prim's algorithm and Kruskal's algorithm, both of which rely on the cut property.

2.4 Prim's Algorithm

Prim's algorithm, proposed by Jarník (1930) and rediscovered by Prim (1957) and Dijkstra (1959), grows a single tree from a chosen start vertex. At every step it adds the cheapest edge that connects a vertex inside the tree to a vertex outside it. The algorithm terminates when all vertices are included. Its running time depends on the data structure used to find the cheapest crossing edge: a simple array gives $O(V^2)$, a binary heap gives $O(E \log V)$, and a Fibonacci heap gives $O(E + V \log V)$.

2.5 Kruskal's Algorithm

Kruskal's algorithm sorts all edges by weight and adds them one at a time, skipping any edge that would create a cycle, using a disjoint-set (union–find) structure. It runs in $O(E \log E)$ time. It performs well on sparse graphs but must sort all $N(N - 1)/2$ edges in a complete graph, making it less attractive than array-based Prim's for dense sensor networks.

2.6 Why Prim's Algorithm for IoT Topologies

In a sensor field the graph is complete, so $E = \Theta(N^2)$. For such dense graphs the array-based Prim's algorithm runs in $\Theta(N^2)$ time, which is linear in the size of the input (the adjacency matrix itself has N^2 entries). Heap-based Prim's and Kruskal's algorithms both cost $\Theta(N^2 \log N)$ here. Prim's algorithm also naturally grows the tree outward from the gateway, which matches how data flows in a sensor network and makes the construction easy to visualise step by step.

3. PROBLEM FORMULATION

3.1 Graph Model

The sensor network is modelled as a complete, undirected, weighted graph G = (V, E, w):

- **Vertices V = {S0, S1, …, S(N−1)}:** each vertex is an IoT sensor node placed on a 2-D canvas.
- **Gateway:** the start node selected by the user (S0 by default), shown in purple in the GUI.
- **Edges E:** a link exists between every pair of sensors, so |E| = N(N − 1)/2.
- **Weight w(u, v):** the transmission power in milliwatts (mW) needed for nodes u and v to communicate, derived from the distance between them on the canvas. Weights are symmetric: w(u, v) = w(v, u).

3.2 Objective Function

Find a spanning tree T ⊆ E such that every sensor is connected to the gateway and the total transmission power

$$P(T) = \Sigma\, w(u, v) \quad \text{for all } (u, v) \in T$$

is minimised, subject to |T| = N − 1 and T containing no cycles.

3.3 Baseline for Comparison

The baseline is the direct-link (star) topology, in which every sensor communicates directly with the gateway. Its cost is

$$P_direct = \Sigma\, w(g, v) \quad \text{for all } v \in V, v \neq g$$

where g is the gateway. The energy saved by the MST topology is P_direct − P(T), and the percentage saving is (P_direct − P(T)) / P_direct × 100.

3.4 Input and Output

Item	Description
Input	Set of N sensor nodes with 2-D coordinates; selected gateway (start node).
Derived input	N × N symmetric adjacency matrix of transmission-power costs (mW).
Output	N − 1 links forming the minimum-power spanning tree, in the order selected.
Metrics	Optimal MST power, direct-link power, energy saved (mW and %).
Per-step output	Chosen minimum link (R1), two alternate candidate links (R2, R3) and their power penalty.

4. ALGORITHM DESIGN

4.1 Design Strategy: Greedy Method

Prim's algorithm follows the greedy paradigm. At each step it makes the locally optimal choice — the cheapest link from the current tree to any unconnected sensor — and never revisits that choice. For the MST problem this local choice is provably globally optimal because of the cut property (Section 4.5).

4.2 Data Structures

- **cost[N][N]:** adjacency matrix holding the transmission power between every pair of sensors.
- **key[N]:** key[v] is the cheapest known link power from the current tree to sensor v; initialised to ∞, and key[start] = 0.
- **parent[N]:** parent[v] is the tree node through which v is connected at the cost key[v].
- **inMST[N]:** boolean array marking sensors already added to the tree.

4.3 Algorithm Steps

1. Initialise key[v] = ∞, parent[v] = −1 and inMST[v] = false for every sensor v. Set key[start] = 0.
2. Repeat the following three steps N times:
3. Select the sensor u not yet in the tree with the smallest key[u] (linear scan, O(N)).
4. Mark u as included: inMST[u] = true. If parent[u] ≠ −1, record the link (parent[u], u) with power key[u].
5. For every sensor v not in the tree, if cost[u][v] < key[v], set key[v] = cost[u][v] and parent[v] = u (O(N)).
6. Return the recorded N − 1 links and their total power.

4.4 Pseudocode

```
PRIM-MST(cost[0..N-1][0..N-1], start)
  for v = 0 to N-1
      key[v] ← ∞ ;  parent[v] ← -1 ;  inMST[v] ← FALSE
  key[start] ← 0
  total ← 0 ;  T ← ∅
  for count = 1 to N
      u ← vertex with minimum key[u] among inMST[u] = FALSE   // O(N)
      inMST[u] ← TRUE
      if parent[u] ≠ -1 then
          T ← T ∪ {(parent[u], u)} ;  total ← total + key[u]
      for v = 0 to N-1                                    // O(N)
          if inMST[v] = FALSE and cost[u][v] < key[v] then
              key[v] ← cost[u][v] ;  parent[v] ← u
  return T, total
```

4.5 Proof of Correctness (Cut Property)

Cut property: For any partition (cut) of the vertices into two non-empty sets S and V − S, if e is the minimum-weight edge crossing the cut, then e belongs to some MST of G.

Proof sketch: Suppose an MST T* does not contain e = (u, v), where u ∈ S and v ∈ V − S. Adding e to T* creates a cycle, and this cycle must cross the cut at least once more through some edge e′. Since w(e) ≤ w(e′), replacing e′ with e gives a spanning tree T* − e′ + e whose weight is no greater than that of T*. Hence that tree is also an MST and it contains e.

Application to Prim's algorithm: At every iteration, Prim's algorithm takes S as the set of sensors already in the tree and adds the cheapest edge crossing (S, V − S). By the cut property, every edge it adds belongs to an MST. By induction on the number of edges, the final tree of N − 1 edges is a Minimum Spanning Tree.

4.6 Alternate Path Penalty

To make each greedy decision transparent, the visualizer also reports the two next-best candidate links at every step (shown as R2 and R3 in orange). The penalty of an alternate link is its power minus the power of the chosen link (R1). For example, at Step 1 the chosen link S0→S1 costs 27.20 mW, while the alternate S0→S3 costs 32.80 mW, so choosing it would add a penalty of 5.60 mW. This feature helps students see why the greedy choice is the best one at that moment.

5. SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Architecture

The application follows a layered design that separates the graph model, the algorithm engine and the user interface:

- **Graph Model Layer:** stores sensor nodes (ID and canvas coordinates) and builds the N × N transmission-power matrix whenever the topology changes.
- **Algorithm Engine:** implements Prim's $O(N^2)$ algorithm as a resumable, step-wise process so the GUI can advance one greedy selection at a time or run all steps at once.
- **Visualisation Layer:** renders nodes, candidate links, chosen MST links and labels on a zoomable canvas, and updates the route-comparison and metrics panels after every step.
- **Control Layer:** handles button events, start-node selection, add/delete modes and viewport commands.

5.2 Development Environment

Component	Details
Programming language	Python 3
GUI toolkit	Tkinter (desktop window with Canvas widget)
Algorithm	Prim's MST, adjacency-matrix / key-array version, $O(N^2)$
Platform	Windows desktop
Units	Transmission power in milliwatts (mW)

5.3 GUI Modules

5.3.1 Topology Builder

- **+ Add Random Sensor:** places a new sensor at a random position on the canvas and assigns the next ID.
- **Delete Node (toggle):** when ON, clicking a sensor removes it and its links from the graph.
- **Load Preset Network:** loads a fixed six-node network (S0–S5) used for demonstration and repeatable testing.
- **Clear Grid:** removes all sensors and resets the algorithm state.

5.3.2 Algorithm Controls

- **Start Node:** drop-down to select the gateway node from which Prim's algorithm grows the tree.
- **Step-by-Step (Evaluate):** performs one iteration of Prim's algorithm and highlights the newly added link.

- **Run All Steps:** completes the remaining iterations and displays the final MST.

5.3.3 Viewport Controls

Zoom In (+), Zoom Out (−) and Fit View to Screen let the user inspect large or dense topologies without losing the overall layout.

5.3.4 Route Comparison and Alternate Path Penalty Panel

The panel at the top of the window shows three cards after each step: R1 (green) is the optimal minimum route chosen by Prim's algorithm; R2 and R3 (orange) are alternate candidate links with their extra power cost relative to R1.

5.3.5 Energy Savings Metrics

The metrics panel reports the Optimal MST power, the Direct Links power (star topology from the gateway) and the Energy Saved in both milliwatts and percent. A status line shows the current mode, the step number (k/N−1) and the number of connected nodes.

5.4 Visual Encoding

Visual element	Meaning
Purple node	Gateway / start node
Blue node	Sensor not yet connected to the tree
Green node	Sensor connected to the MST
Thick green line	Link selected in the MST (labelled with power in mW)
Dotted orange line	Alternate candidate link (R2 / R3) at the current step
Faint dashed grey line	Other possible links in the complete graph

5.5 Core Implementation

The algorithm engine stores its state between steps so that each button press performs exactly one greedy selection. The core logic is shown below.

```
import math

def build_cost_matrix(nodes, scale):
    n = len(nodes)
    cost = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            (x1, y1), (x2, y2) = nodes[i], nodes[j]
            d = math.hypot(x2 - x1, y2 - y1)
            cost[i][j] = cost[j][i] = round(d * scale, 2)   # mW
    return cost

def prim_mst(cost, start=0):
    n = len(cost)
    INF = float("inf")
```

```
    key, parent, in_mst = [INF] * n, [-1] * n, [False] * n
    key[start] = 0
    edges, total = [], 0.0
    for _ in range(n):
        u = min((v for v in range(n) if not in_mst[v]),
                key=lambda v: key[v])                # O(N) selection
        in_mst[u] = True
        if parent[u] != -1:
            edges.append((parent[u], u, cost[parent[u]][u]))
            total += cost[parent[u]][u]
        for v in range(n):                            # O(N) key update
            if not in_mst[v] and cost[u][v] < key[v]:
                key[v], parent[v] = cost[u][v], u
    return edges, total

def direct_cost(cost, gateway=0):
    return sum(cost[gateway][v] for v in range(len(cost)) if v != gateway)
```

6. COMPLEXITY ANALYSIS

6.1 Time Complexity

Let N be the number of sensor nodes. The array-based implementation performs the following work:

Operation	Cost per iteration	Total (N iterations)
Build cost matrix	—	$O(N^2)$
Initialise key, parent, inMST	—	$O(N)$
Find minimum-key unvisited node	$O(N)$	$O(N^2)$
Update keys of neighbours	$O(N)$	$O(N^2)$
Overall		$O(N^2)$

The outer loop runs N times, and each iteration does two linear scans over N vertices. Hence $T(N) = N \cdot (cN + cN) = \Theta(N^2)$. This bound holds in the best, average and worst cases, because the scans always examine every vertex regardless of edge weights.

Since the input itself — the adjacency matrix of a complete graph — has N^2 entries, any algorithm must read $\Theta(N^2)$ values. The array-based Prim's algorithm is therefore asymptotically optimal for this setting.

6.2 Space Complexity

- **Adjacency matrix:** $O(N^2)$.
- **key, parent and inMST arrays:** $O(N)$.
- **MST edge list:** $N - 1$ edges, $O(N)$.

Total auxiliary space is $O(N^2)$, dominated by the cost matrix. Excluding the input matrix, the algorithm needs only $O(N)$ extra space.

6.3 Comparison with Alternative Approaches

For a complete graph, $E = N(N - 1)/2 = \Theta(N^2)$. Substituting into the standard bounds gives:

Algorithm / data structure	General bound	Complete graph ($E = \Theta(N^2)$)
Prim's – array (this project)	$O(V^2)$	$O(N^2)$
Prim's – binary heap	$O(E \log V)$	$O(N^2 \log N)$
Prim's – Fibonacci heap	$O(E + V \log V)$	$O(N^2)$, high constant factors
Kruskal's – sort + union-find	$O(E \log E)$	$O(N^2 \log N)$
Brute force (all spanning trees)	$O(N^{(N-2)} \cdot N)$	Exponential (Cayley's formula)

The array version matches the Fibonacci-heap bound on dense graphs while being far simpler and faster in practice. Brute force is infeasible: by Cayley's formula a complete graph on just 10 sensors has 10^8 spanning trees.

6.4 Growth of Operations

Nodes (N)	Edges N(N−1)/2	Prim array ≈ 2N²	Kruskal ≈ E log₂E
6	15	72	≈ 59
50	1,225	5,000	≈ 12,560
100	4,950	20,000	≈ 60,750
500	124,750	500,000	≈ 2,112,000
1,000	499,500	2,000,000	≈ 9,454,000

The counts are approximate and ignore constant factors, but they show the gap widening with N: for large, dense sensor fields the array-based Prim's algorithm performs several times fewer basic operations than Kruskal's algorithm.

7. RESULTS AND DISCUSSION

This chapter walks through a full run of the visualizer on the six-node preset network (S0–S5) with S0 as the gateway. Each figure corresponds to one press of the Step-by-Step (Evaluate) button.

7.1 Initial Topology

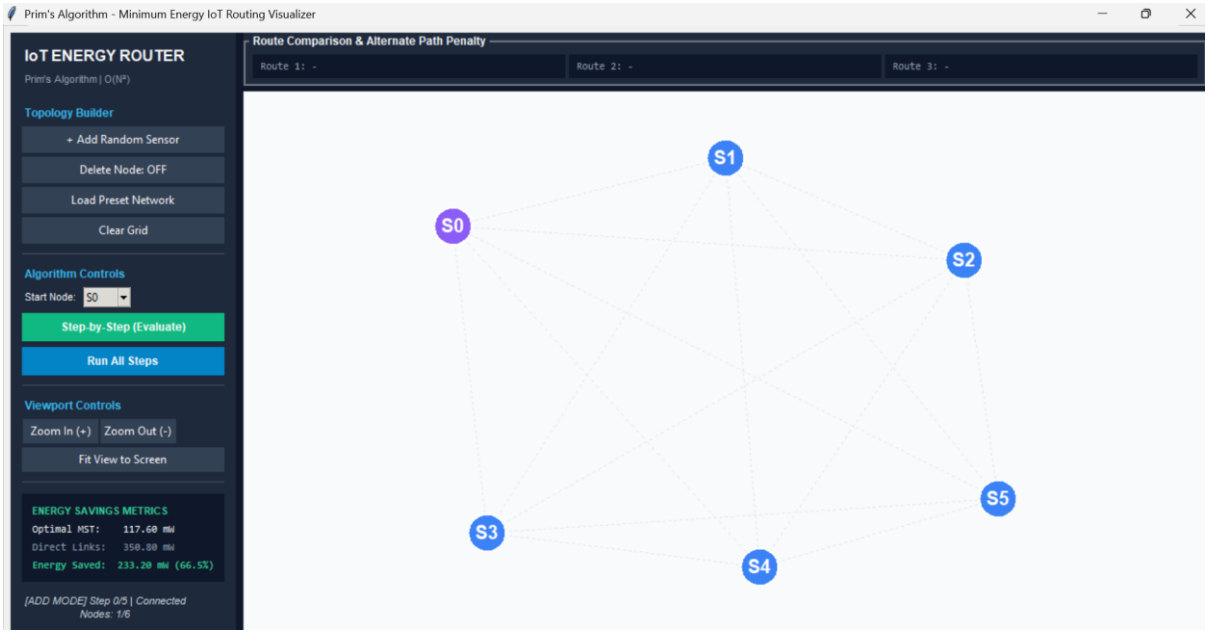


Figure 7.1: Preset six-node network before execution (Step 0/5, 1 of 6 nodes connected)

The canvas shows the gateway S0 in purple and five unconnected sensors in blue. Faint dashed lines indicate that every pair of sensors can communicate, forming a complete graph with 15 candidate links. The route-comparison panel is empty because no step has been evaluated yet.

7.2 Step-wise Construction of the MST

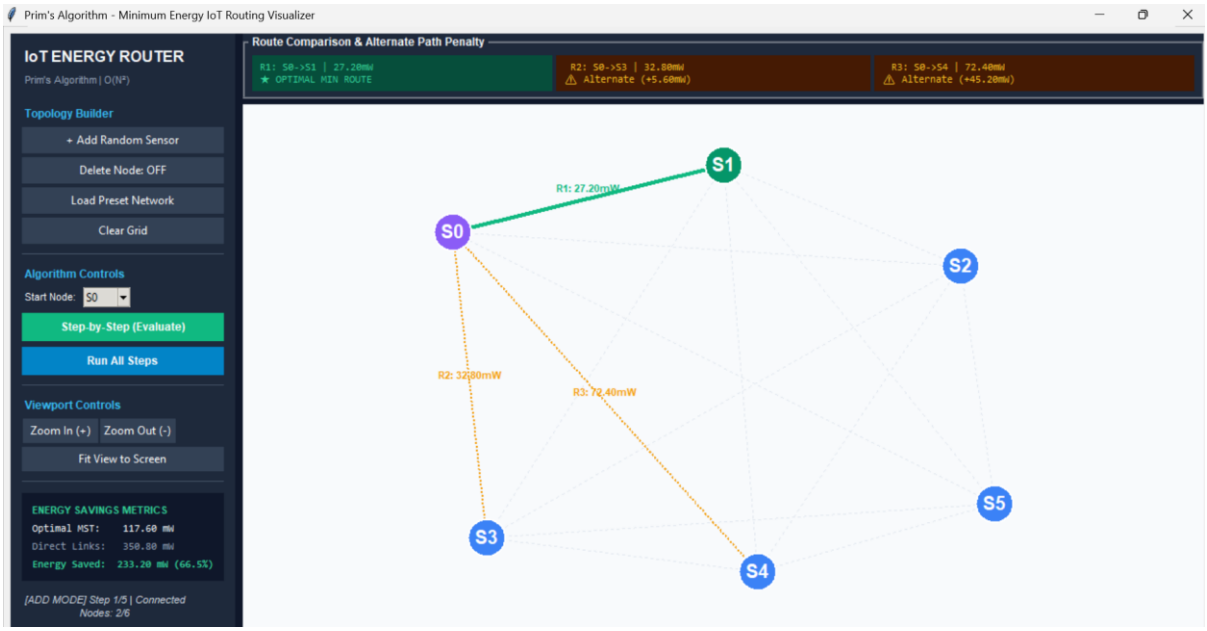


Figure 7.2: Step 1 – link S0→S1 (27.20 mW) selected

Starting from S0, the cheapest outgoing link is S0→S1 at 27.20 mW. The alternatives S0→S3 (32.80 mW) and S0→S4 (72.40 mW) would cost 5.60 mW and 45.20 mW more respectively.

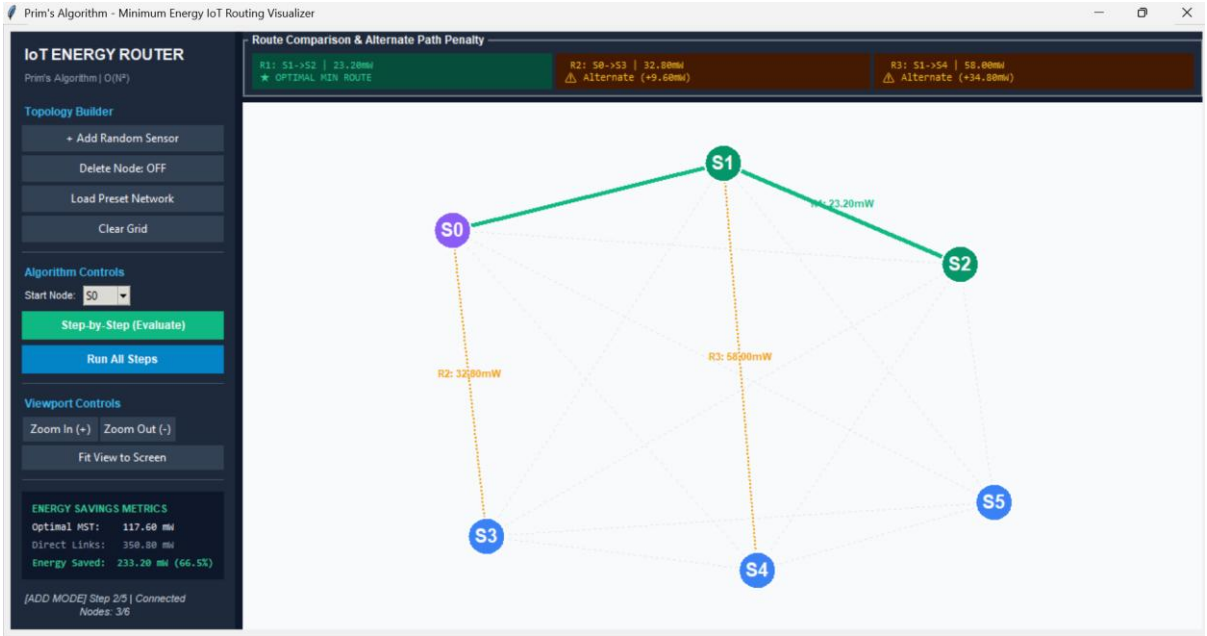


Figure 7.3: Step 2 – link S1→S2 (23.20 mW) selected

With the tree {S0, S1}, the cheapest crossing link is S1→S2 at 23.20 mW. Note that the algorithm considers links from every tree node, not just the most recent one.

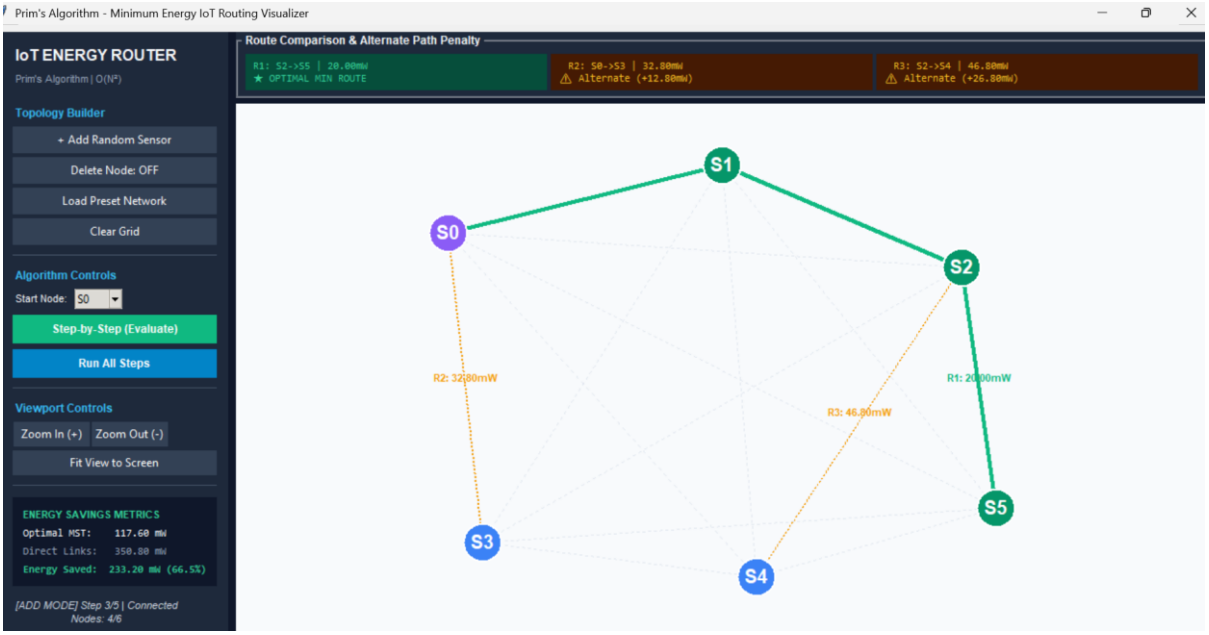


Figure 7.4: Step 3 – link S2→S5 (20.00 mW) selected

The tree grows along the right side of the field. S2→S5 at 20.00 mW is the cheapest link in the entire MST.

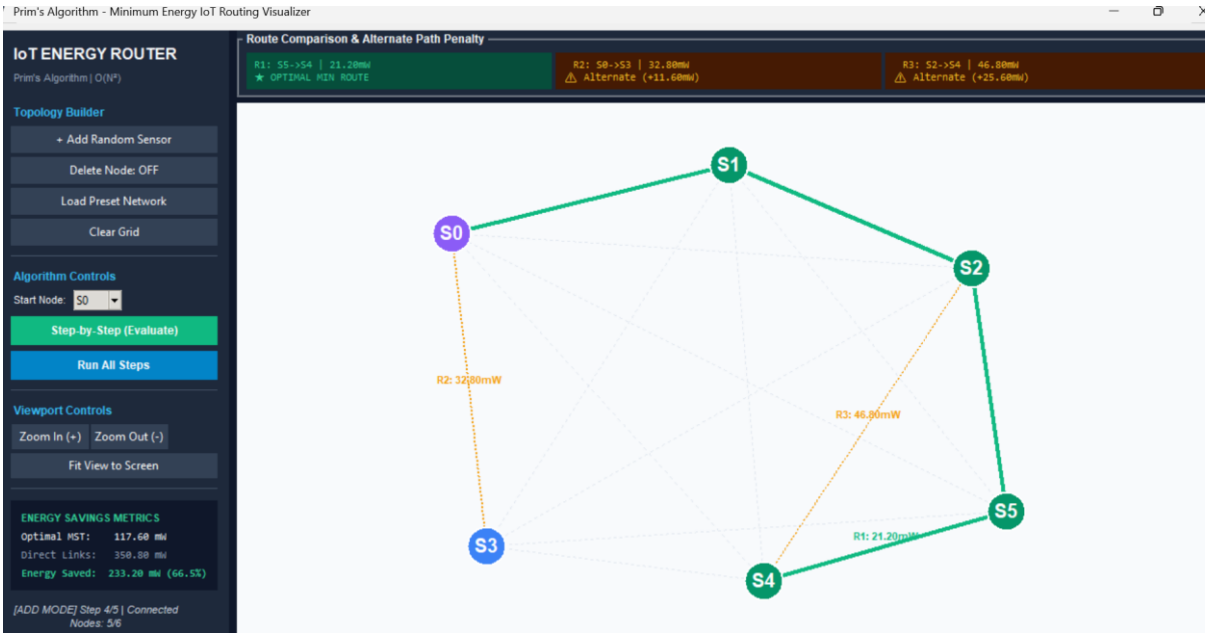


Figure 7.5: Step 4 – link S5→S4 (21.20 mW) selected

S4 is reached through S5 at 21.20 mW, which is far cheaper than the direct gateway link S0→S4 (72.40 mW) seen in Step 1.

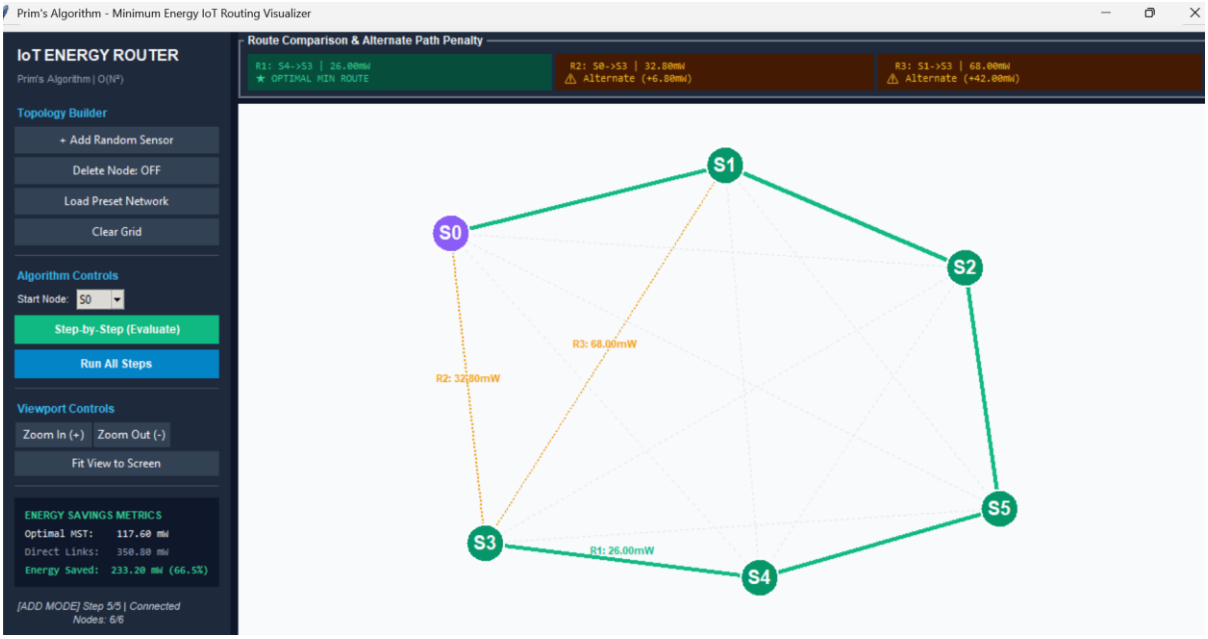


Figure 7.6: Step 5 – link S4→S3 (26.00 mW) selected; MST complete (6 of 6 nodes connected)

In the final step S3 is connected via S4 at 26.00 mW. The direct gateway link S0→S3 (32.80 mW), which appeared as an alternate in every step, is never chosen because a cheaper relay path always existed.

7.3 Execution Trace

Table 7.1: Step-wise trace of Prim's algorithm on the preset network (start node S0)

Step	Chosen link (R1)	Power (mW)	Alternates R2 / R3 (penalty)	Cumulative (mW)	Connected
1	S0 → S1	27.20	S0→S3 (+5.60), S0→S4 (+45.20)	27.20	2 / 6
2	S1 → S2	23.20	S0→S3 (+9.60), S1→S4 (+34.80)	50.40	3 / 6
3	S2 → S5	20.00	S0→S3 (+12.80), S2→S4 (+26.80)	70.40	4 / 6
4	S5 → S4	21.20	S0→S3 (+11.60), S2→S4 (+25.60)	91.60	5 / 6
5	S4 → S3	26.00	S0→S3 (+6.80), S1→S3 (+42.00)	117.60	6 / 6

7.4 Energy Savings

Table 7.2: Energy comparison for the preset network

Metric	Value
Optimal MST transmission power	117.60 mW
Direct-link (star) transmission power	350.80 mW
Energy saved	233.20 mW
Percentage saving	66.5 %

The final topology is a chain S0 – S1 – S2 – S5 – S4 – S3 that follows the perimeter of the sensor field. Every hop is short, so the total power falls to roughly one-third of the star topology. In a real deployment this would translate directly into longer battery life for the network as a whole.

7.5 Discussion

- The total of the five chosen links (27.20 + 23.20 + 20.00 + 21.20 + 26.00) equals the Optimal MST value of 117.60 mW shown in the metrics panel, confirming the implementation's bookkeeping.
- The selected link weights are not monotonically increasing (27.20, 23.20, 20.00, 21.20, 26.00), which distinguishes Prim's algorithm from Kruskal's: Prim's always picks the cheapest link crossing the current cut, not the globally next-cheapest edge.
- The alternate-penalty panel shows that the greedy choice was strictly better than every displayed alternative at every step, illustrating the cut property in action.
- A trade-off of the MST is hop count: S3 is five hops from the gateway, which can increase latency. This is discussed under limitations.

8. TESTING

8.1 Testing Approach

The application was tested at two levels: functional testing of the GUI controls and correctness testing of the algorithm output against hand-computed values on the preset network.

8.2 Test Cases

ID	Test case	Expected result	Status
TC1	Load Preset Network	Six nodes S0–S5 appear; status shows Step 0/5, 1/6 connected	Pass
TC2	Step-by-Step on preset (×5)	Links added in order S0→S1, S1→S2, S2→S5, S5→S4, S4→S3	Pass
TC3	MST total on preset	Optimal MST = 117.60 mW	Pass
TC4	Energy-saving metrics	Direct = 350.80 mW; Saved = 233.20 mW (66.5%)	Pass
TC5	Run All Steps	Same final MST and total as five single steps	Pass
TC6	Change start node	MST total unchanged (117.60 mW); construction order differs	Pass
TC7	Add Random Sensor	New node with next ID added; matrix and metrics recomputed	Pass
TC8	Delete Node mode	Clicked node and its links removed; IDs and drop-down updated	Pass
TC9	Clear Grid then run	No crash; algorithm does nothing on an empty graph	Pass
TC10	Single-node graph	MST has 0 edges; total = 0 mW	Pass
TC11	Two-node graph	MST is the single link between them	Pass
TC12	Zoom In / Out / Fit View	Canvas scales and re-centres without altering results	Pass

8.3 Correctness Verification

For the preset network, the MST was verified by checking that the five chosen edges sum to the reported total, that they connect all six nodes without a cycle, and that at each step no crossing edge displayed was cheaper than the chosen one. Test TC6 relies on a known property of MSTs: when all edge weights are distinct the MST is unique, so its total weight must be the same regardless of the start node.

9. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

9.1 Applications

- **Precision agriculture:** connecting soil-moisture and weather sensors spread across a farm with minimum radio power.
- **Smart cities:** planning backbones for street-light controllers, parking sensors and air-quality monitors.
- **Structural health monitoring:** linking vibration and strain sensors on bridges and buildings.
- **Industrial IoT:** low-power telemetry networks in factories and warehouses.
- **Environmental and disaster monitoring:** forest-fire, flood and landslide sensor fields where battery replacement is difficult.
- **Education:** an interactive teaching aid for greedy algorithms and Minimum Spanning Trees.

9.2 Limitations

- The MST minimises total power, not the maximum load on any single node; relay nodes near the gateway may drain faster.
- Long chains (such as S0 → … → S3 in the preset) increase hop count and end-to-end latency.
- A tree has no redundancy: if one relay node fails, the sensors downstream are disconnected.
- Edge weights are static and based on distance only; real links are affected by obstacles, interference and fading.
- Residual battery energy, data rate and duty cycling are not modelled.

9.3 Future Enhancements

- Add a heap-based Prim's and Kruskal's implementation with a live timing comparison to demonstrate the complexity results empirically.
- Introduce degree-constrained or hop-bounded MSTs to balance energy with latency and relay load.
- Incorporate residual battery levels into edge weights and rebuild the tree dynamically as nodes deplete.
- Support k-connected topologies for fault tolerance.
- Import real sensor coordinates from CSV files and export the resulting topology and metrics.
- Simulate packet flow to estimate network lifetime under the MST and star topologies.

10. CONCLUSION

This project applied Prim's algorithm to a practical and important problem: building an energy-efficient communication backbone for an IoT sensor network. By modelling sensors as vertices of a complete graph and transmission power as edge weights, the task of minimising total network power reduces exactly to finding a Minimum Spanning Tree.

The array-based Prim's algorithm was chosen because it runs in $\Theta(N^2)$ time, which is optimal for dense graphs and better than heap-based Prim's or Kruskal's algorithms in this setting. Its correctness follows from the cut property, and the visualizer makes this property visible by showing the chosen link alongside alternate links and their power penalty at every step.

The IoT Energy Router GUI lets users build topologies, run the algorithm step by step and see energy metrics update live. On the six-node preset network it reduced total transmission power from 350.80 mW to 117.60 mW, a saving of 66.5%. The project demonstrates how a classical greedy algorithm, analysed carefully, can deliver substantial real-world benefits in energy-constrained systems.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2] E. Horowitz, S. Sahni and S. Rajasekaran, Fundamentals of Computer Algorithms, 2nd ed. Universities Press, 2008.

[3] J. Kleinberg and É. Tardos, Algorithm Design. Pearson, 2006.

[4] R. C. Prim, "Shortest connection networks and some generalizations," Bell System Technical Journal, vol. 36, no. 6, pp. 1389–1401, 1957.

[5] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48–50, 1956.

[6] W. B. Heinzelman, A. P. Chandrakasan and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," Proc. 33rd Hawaii International Conference on System Sciences, 2000.

[7] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam and E. Cayirci, "Wireless sensor networks: a survey," Computer Networks, vol. 38, no. 4, pp. 393–422, 2002.

[8] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3 Documentation.